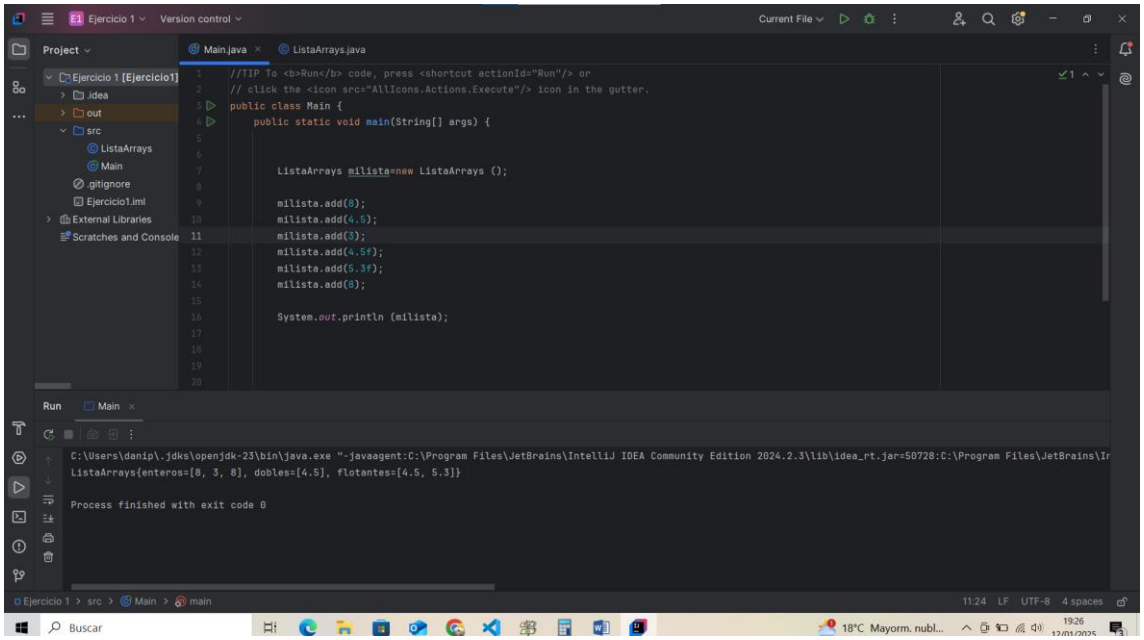
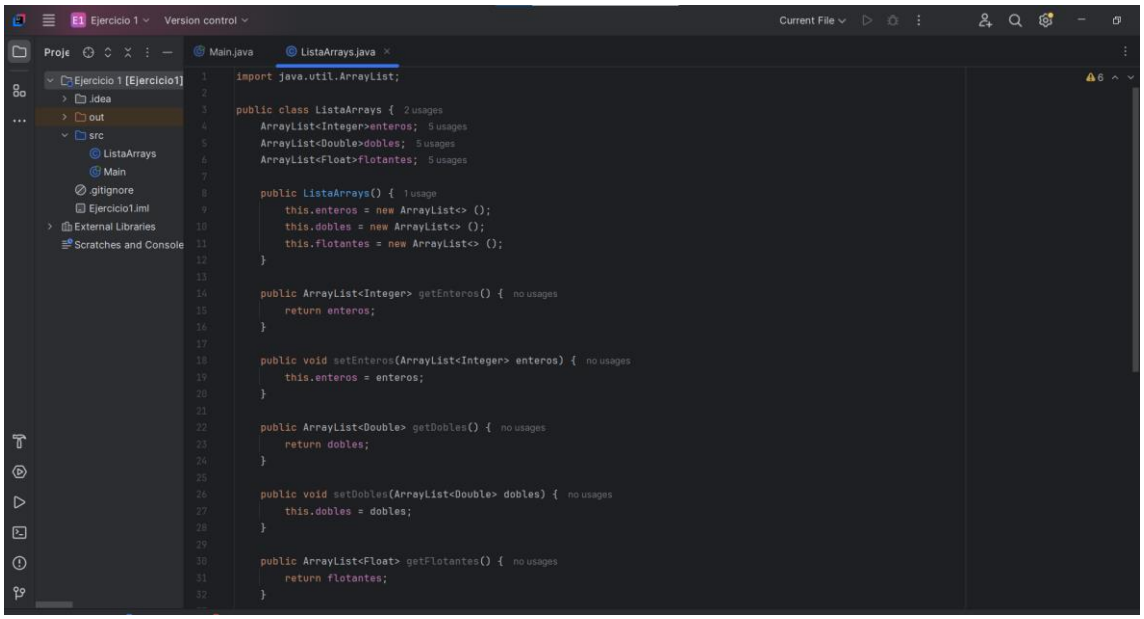
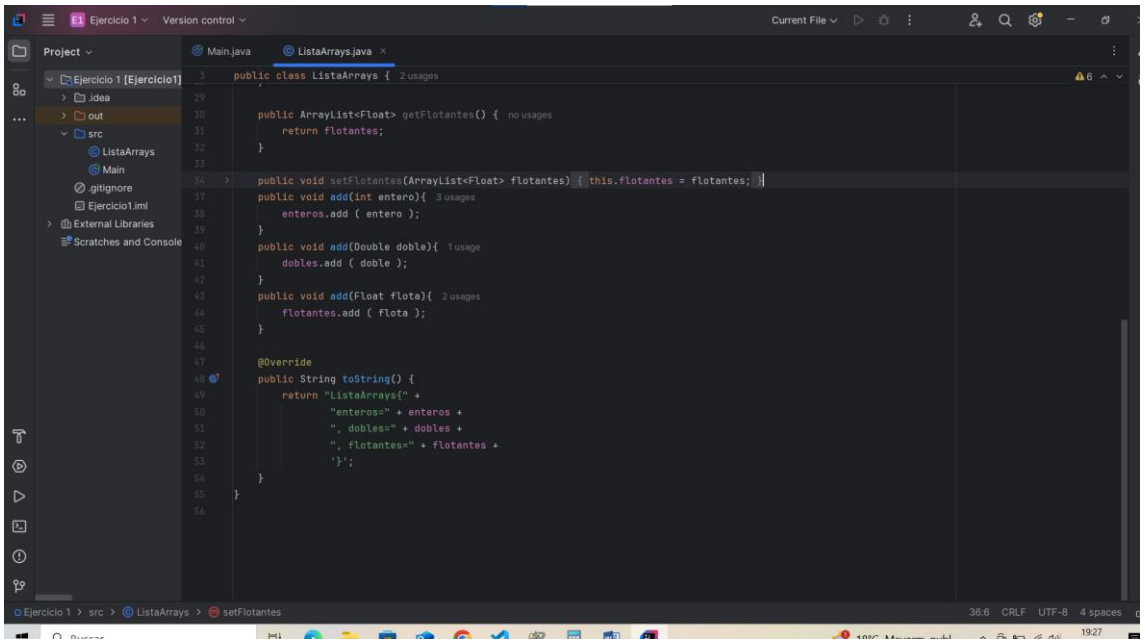
EJERCICIO 1


Ejercicio 1
Version control
Current File
Project
Main.java
ListaArrays.java
Ejercicio 1 [Ejercicio1]
.idea
out
src
ListaArrays
Main
.gitignore
Ejercicio1.iml
External Libraries
Scratches and Console
//TIP To Run code, press <shortcut actionId="Run"/> or
// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.
public class Main {
public static void main(String[] args) {
ListaArrays milista=new ListaArrays ();
milista.add(8);
milista.add(4.5);
milista.add(3);
milista.add(4.5f);
milista.add(5.3f);
milista.add(8);
System.out.println (milista);
Run
Main
C:\Users\danip\.jdks\openjdk-23\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.2.3\lib\idea_rt.jar=50728:C:\Program Files\JetBrains\Ir
ListaArrays{enteros=[8, 3, 8], dobles=[4.5], flotantes=[4.5, 5.3]}
Process finished with exit code 0
Ejercicio 1 > src > Main > main
11:24
LF
UTF-8
4 spaces
Buscar
18°C Mayorm. nubl...
19:26
12/01/2025


Ejercicio 1
Version control
Current File
Main.java
ListaArrays.java
Ejercicio 1 [Ejercicio1]
.idea
out
src
ListaArrays
Main
.gitignore
Ejercicio1.iml
External Libraries
Scratches and Console
import java.util.ArrayList;
public class ListaArrays { 2 usages
ArrayList<Integer>enteros; 5 usages
ArrayList<Double>dobles; 5 usages
ArrayList<Float>flotantes; 5 usages
public ListaArrays() { 1 usage
this.enteros = new ArrayList<> ();
this.dobles = new ArrayList<> ();
this.flotantes = new ArrayList<> ();
}
public ArrayList<Integer> getEnteros() { no usages
return enteros;
}
public void setEnteros(ArrayList<Integer> enteros) { no usages
this.enteros = enteros;
}
public ArrayList<Double> getDobles() { no usages
return dobles;
}
public void setDobles(ArrayList<Double> dobles) { no usages
this.dobles = dobles;
}
public ArrayList<Float> getFlotantes() { no usages
return flotantes;
}



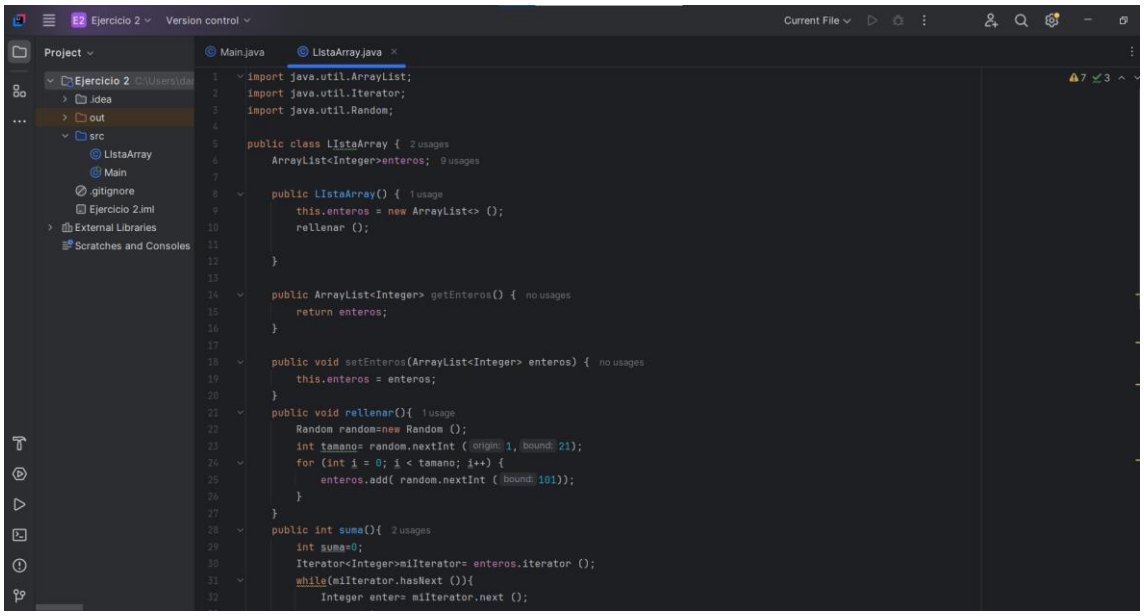
```java
public class ListaArrays {

    public ArrayList<Float> getFlotantes() {
        return flotantes;
    }

    public void setFlotantes(ArrayList<Float> flotantes) { this.flotantes = flotantes; }
    public void add(int entero){
        enteros.add ( entero );
    }
    public void add(Double doble){
        dobles.add ( doble );
    }
    public void add(Float flota){
        flotantes.add ( flota );
    }

    @Override
    public String toString() {
        return "ListaArrays{" +
                "enteros=" + enteros +
                ", dobles=" + dobles +
                ", flotantes=" + flotantes +
                '}';
    }
}
```

EJERCICIO 2



```java
import java.util.ArrayList;
import java.util.Iterator;
import java.util.Random;

public class LIstaArray {
    ArrayList<Integer>enteros;

    public LIstaArray() {
        this.enteros = new ArrayList<> ();
        rellenar ();

    }

    public ArrayList<Integer> getEnteros() {
        return enteros;
    }

    public void setEnteros(ArrayList<Integer> enteros) {
        this.enteros = enteros;
    }
    public void rellenar(){
        Random random=new Random ();
        int tamano= random.nextInt ( origin: 1, bound: 21);
        for (int i = 0; i < tamano; i++) {
            enteros.add( random.nextInt ( bound: 101));
        }
    }
    public int suma(){
        int suma=0;
        Iterator<Integer>miIterator= enteros.iterator ();
        while(miIterator.hasNext ()){
            Integer enter= miIterator.next ();
```

```java
    public int suma(){  2 usages
        int suma=0;
        Iterator<Integer>miIterator= enteros.iterator ();
        while(miIterator.hasNext ()){
            Integer enter= miIterator.next ();
            suma+=enter;

        }return suma;
    }
    public double media(){  1 usage
        double media=suma()/enteros.size ();
        return media;
    }
    public int numeroMax(){  1 usage
        int numeroMax=0;
        Iterator<Integer>miIterator= enteros.iterator ();
        while(miIterator.hasNext ()) {
            Integer enter = miIterator.next ();
            if(enter>numeroMax){
                numeroMax=enter;
            }
        }return numeroMax;

    }
    public int numeroMin(){  1 usage
        int numeroMin=Integer.MAX_VALUE;
        Iterator<Integer>miIterator= enteros.iterator ();
        while(miIterator.hasNext ()) {
            Integer enter = miIterator.next ();
```

```java
    }
    public int numeroMin(){  1 usage
        int numeroMin=Integer.MAX_VALUE;
        Iterator<Integer>miIterator= enteros.iterator ();
        while(miIterator.hasNext ()) {
            Integer enter = miIterator.next ();
            if (enter < numeroMin) {
                numeroMin = enter;

            }
        }return numeroMin;
    }

    @Override
    public String toString() {
        return "LIstaArray{" +
                "enteros=" + enteros +
                '}';
    }
}
```

```java
public class Main {
    public static void main(String[] args){


LIstaArray milista=new LIstaArray ();
        System.out.println (milista);
        System.out.println ("la suma de los números del array es "+milista.suma ());
        System.out.println ("la media de los números del array es  "+milista.media ());
        System.out.println ("El número máximo de la lista es  "+milista.numeroMax ());
        System.out.println ("El número mínimo de la lista es "+milista.numeroMin ());



 }
}
```

Ejercicio 3

```java
import java.util.LinkedList;

public class PilaPalabras {
    LinkedList<String>pila;

    public PilaPalabras() {
        this.pila = new LinkedList<> ();
    }

    public LinkedList<String> getPila() {
        return pila;
    }

    public void setPila(LinkedList<String> pila) {
        thi
    }
    public
        pil
    }
    public
        if (!pila.isEmpty ()) {
            System.out.println ( "La palabra " + pila.getFirst () + " ha sido eliminada" );
            pila.removeFirst ();
        } else {
            System.out.println ( "La pila está vacía" );
        }
    }
    public void mostrarPila(){
        System.out.println("Las palabras de la pila son ");
        if(!pila.isEmpty ()){
        for (int i = 0; i < pila.size (); i++) {
            System.out.println ( pila.get ( i ) );
```

```java
public class PilaPalabras {
    public void mostrarPila(){
        }
        }else{
            System.out.println ("No hay palabras que mostrar");
        }

    }
    public void obtenerPalabra(){
        if(!pila.isEmpty ()) {
            System.out.println ( "La palabra en la cima de la pila es " + pila.getFirst () );
        }else{
            System.out.println ("La pila está vacia");
        }
    }

    public void pilaPalabrasVacia(){
        if(pila.isEmpty ()){
            System.out.println ("La pila está vacia");
        }else{
            System.out.println ("La pila no está vacia: ");
            mostrarPila ();
        }
    }

    @Override
    public String toString() {
        return "PilaPalabras{" +
                "pila=" + pila +
                '}';
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        /*
        Escribir una clase, de nombre PilaPalabras, para gestionar una estructura de pila que permita apilar
         y desapilar objetos de la clase String. La clase implementará el método apilarPalapra para poner una
         palabra encima de la pila, el método desapilarPalabra para quitar el elemento de la cima de la pila de
         devolviéndolo y el método obtenerPalabra para obtener la palabra situada en la cima de la pila sin
         quitarla de ella. También implementará el método pilaPalabrasVacia para determinar si la pila está o
         no vacia. Los métodos deben implementarse utilizando la clase LinkedList
         */
PilaPalabras mipila=new PilaPalabras ();

        mipila.apilarPalbra ( palabra: "casa");
        mipila.apilarPalbra ( palabra: "ventana");
        mipila.apilarPalbra ( palabra: "puerta");
        mipila.apilarPalbra ( palabra: "cocina");
        mipila.apilarPalbra ( palabra: "salón" );
        mipila.mostrarPila ();
        mipila.desapilarPalabra ();
        mipila.mostrarPila ();
        mipila.obtenerPalabra ();
        mipila.pilaPalabrasVacia ();

    }
}
```

```
public class Main {
        mipila.mostrarPila ();
        mipila.desapilarPalabra ();
        mipila.mostrarPila ();
```

```
C:\Users\danip\.jdks\openjdk-23\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.2.3\lib\idea_rt.jar=51837:C:\Program Files\JetBrains\Ir
Las palabras de la pila son
salón
cocina
puerta
ventana
casa
La palabra salón ha sido eliminada
Las palabras de la pila son
cocina
puerta
ventana
casa
La palabra en la cima de la pila es cocina
La pila no está vacía:
Las palabras de la pila son
cocina
puerta
ventana
casa

Process finished with exit code 0
```

EJERCICIO 4

```
import java.util.Objects;

public class Ficha implements Comparable<Ficha>{  8 usages
    int num1;  10 usages
    int num2;  10 usages

    public Ficha(int num1, int num2) {  1 usage
        this.num1 = num1;
        this.num2 = num2;
    }

    public int getNum1() {  no usages
        return num1;
    }

    public void setNum1(int num1) {  no usages
        this.num1 = num1;
    }

    public int getNum2() {  no usages
        return num2;
    }
;
    public void setNum2(int num2) {  no usages
        this.num2 = num2;
    }
    public int pesoPonderado(){  2 usages
        int numeroMax=0;
        int numeroMin=0;
        if(num1>num2){
            numeroMax=num1;
            numeroMin=num2;
```

```java
public class Ficha implements Comparable<Ficha>{ 8 usages
    }
    public int pesoPonderado(){ 2 usages
        int numeroMax=0;
        int numeroMin=0;
        if(num1>num2){
            numeroMax=num1;
            numeroMin=num2;
        }else{
            numeroMax=num2;
            numeroMin=num1;
        }
        return numeroMax*6+numeroMin;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass () != o.getClass ()) return false;
        Ficha ficha = (Ficha) o;
        return num1 == ficha.num1 && num2 == ficha.num2;
    }

    @Override
    public int hashCode() {
        return Objects.hash ( num1, num2 );
    }

    @Override
    public int compareTo(Ficha o) {
        return Integer.compare ( pesoPonderado (),o.pesoPonderado () );
    }
```

```java
public class Ficha implements Comparable<Ficha>{ 8 usages
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass () != o.getClass ()) return false;
        Ficha ficha = (Ficha) o;
        return num1 == ficha.num1 && num2 == ficha.num2;
    }

    @Override
    public int hashCode() {
        return Objects.hash ( num1, num2 );
    }

    @Override
    public int compareTo(Ficha o) {
        return Integer.compare ( pesoPonderado (),o.pesoPonderado () );
    }

    @Override
    public String toString() {
        return "[ "+ num1 +
                " | " + num2 +
                " ] ";
    }
}
```

```java
import java.util.NavigableSet;
import java.util.Random;
import java.util.TreeSet;

public class Tester { 2 usages
    TreeSet<Ficha>domino; 7 usages

    public Tester() { 1 usage
        this.d
        genera
    }
    public Tre
        return domino;
    }

    public void setDomino(TreeSet<Ficha> domino) { no usages
        this.domino = domino;
    }
    public void generarFichas(){ 1 usage
        Random random=new Random ();

        while(domino.size ()<6){
            domino.add ( new Ficha ( random.nextInt ( origin: 1, bound: 7), random.nextInt ( origin: 1, bound: 7) ) );

        }
    }
    public void ordenDescendiente(){ 1 usage
        System.out.println ( domino.descendingSet ());
    }
```

```java
public class Tester {

    public TreeSet<Ficha> getDomino() {
        return domino;
    }

    public void setDomino(TreeSet<Ficha> domino) {
        this.domino = domino;
    }
    public void generarFichas(){
        Random random=new Random ();

        while(domino.size ()<6){
            domino.add ( new Ficha ( random.nextInt ( origin: 1, bound: 7), random.nextInt ( origin: 1, bound: 7) ) );

        }
    }
    public void ordenDescendiente(){
        System.out.println ( domino.descendingSet ());
    }

    @Override
    public String toString() {
        return ""+domino;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        /*
         Implementa una clase Ficha (ficha de dominó) con su constructor.
         Después implementa una clase tester con su constructor y su toString() que genere
          una lista de 6 fichas al azar y las imprima ordenadas ascendentemente y
           descendentemente de acuerdo a un "peso ponderado" (que consistirá en el número
           más alto de la ficha multiplicado por 6 más el otro número).
           Para eso implementa también el método pesoPonderado().
           Realizar el ejercicio con un TreeSet considerando que las fichas del dominó
            no se pueden repetir.
         */
Tester mitester=new Tester ();
        System.out.println ("Fichas en orden ascedente de peso ponderado");
        System.out.println (mitester);
        System.out.println ("Fichas en orden descendiente de peso ponderado");
        mitester.ordenDescendiente ();
    }
}
```

```
C:\Users\danip\.jdks\openjdk-23\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.2.3\lib\idea_rt.jar=52510:C:\Program Files\JetBrains\Ir
Fichas en orden ascedente de peso ponderado
[[ 3 | 2 ] , [ 3 | 3 ] , [ 4 | 3 ] , [ 5 | 3 ] , [ 6 | 2 ] , [ 4 | 6 ] ]
Fichas en orden descendiente de peso ponderado
[[ 4 | 6 ] , [ 6 | 2 ] , [ 5 | 3 ] , [ 4 | 3 ] , [ 3 | 3 ] , [ 3 | 2 ] ]

Process finished with exit code 0
```

Ejercicio 5

```java
import java.util.HashMap;
import java.util.Iterator;
import java.util.Scanner;
import java.util.TreeMap;

public class Tester {
  private TreeMap<String,Persona> milista;//AL PRINCIPIO LO HICE CON UN HASHMAP Y SOLO CAMBIANDO A TREEMAP, HA ORDENADO POR KEYS

    public Tester() {
        this.milista = new TreeMap<> ();
    }

        public void mostrarKey(){
            System.out.println (milista.keySet ());
    }
    public void mostrarValues(){
        System.out.println (milista.values ());
    }

        public void anadirPersonas(String key,String nombre,String apellido){
        milista.put (key,new Persona (  nombre,apellido)) ;
        }
        public void mostrarEntries(){
            System.out.println (milista.entrySet ());
        }
        public void buscarkey(String key){
            System.out.println (milista.get ( key ));
        }
}
```

```java
public class Persona {
    private String nombre;
    private String apellido;

    public Persona(String nombre, String apellido) {
        this.nombre = nombre;
        this.apellido = apellido;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }

    @Override
    public String toString() {
        return
                "Nombre='" + nombre + '\'' +
                ", Apellido='" + apellido + '\'' +
                '}';
    }
```

```java
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {

        Tester mitest=new Tester ();
        mitest.anadirPersonas ( key: "54057854R", nombre: "daniel", apellido: "pulido" );
        mitest.anadirPersonas ( key: "78511652w", nombre: "juan", apellido: "pulido" );//SE HA REPETIDO LA KEY Y SOBREESCRIBE EL ANTERIOR
        mitest.anadirPersonas ( key: "54068859e", nombre: "moneiba", apellido: "ravelo" );

        mitest.mostrarKey ();
        mitest.mostrarValues ();
        mitest.mostrarEntries ();

        mitest.buscarkey ( "54887857e" );

    }
}
```

```java
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {

        Tester mitest=new Tester ();
        mitest.anadirPersonas ( key: "54057854R", nombre: "daniel", apellido: "pulido" );
        mitest.anadirPersonas ( key: "78511643w", nombre: "juan", apellido: "pulido" );//SE HA REPETIDO LA KEY Y SOBREESCRIBE EL AN
        mitest.anadirPersonas ( key: "54068841e", nombre: "moneiba", apellido: "ravelo" );

        mitest.mostrarKey ();
        mitest.mostrarValues ();
        mitest.mostrarEntries ();

        mitest.buscarkey ( "54887857e" );
```

```
C:\Users\danip\.jdks\openjdk-23\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.2.3\lib\idea_rt.jar=52969:C:\Program Fi
[54057854R, 54068841e, 78511643w]
[Nombre='daniel', Apellido='pulido'}, Nombre='moneiba', Apellido='ravelo'}, Nombre='juan', Apellido='pulido'}]
[54057854R=Nombre='daniel', Apellido='pulido'}, 54068841e=Nombre='moneiba', Apellido='ravelo'}, 78511643w=Nombre='juan', Apellido='pulido'}]
null

Process finished with exit code 0
```

Ejercicio 6

```java
import java.util.Objects;

public class Coche implements Comparable<Coche>{ 18 usages
    private String marca; 7 usages
    private int kilometros; 7 usages

    public Coche(String marca, int kilometros) { 1 usage
        this.marca = marca;
        this.kilometros = kilometros;
    }

    public String getMarca() { 1 usage
        return marca;
    }

    public void setMarca(String marca) { no usages
        this.marca = marca;
    }

    public int getKilometros() { 5 usages
        return kilometros;
    }

    public void setKilometros(int kilometros) { no usages
        this.kilometros = kilometros;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass () != o.getClass ()) return false;
        Coche coche = (Coche) o;
```

```java
public class Coche implements Comparable<Coche>{ 18 usages

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass () != o.getClass ()) return false;
        Coche coche = (Coche) o;
        return kilometros == coche.kilometros && Objects.equals ( marca, coche.marca );
    }

    @Override
    public int hashCode() {
        return Objects.hash ( marca, kilometros );
    }

    @Override
    public int compareTo(Coche o) { return Integer.compare ( getKilometros (),o.getKilometros () ); }

    @Override
    public String toString() {
        return "Coche{" +
                "marca='" + marca + '\'' +
                ", kilometros=" + kilometros +
                '}';
    }
}
```

```java
import java.util.HashSet;
import java.util.Iterator;
import java.util.Scanner;
import java.util.TreeSet;

public class LIstaCoches { 2 usages
    private HashSet<Coche>lista; 9 usages

    public LIstaCoches() { this.lista = new HashSet<> (); }

    public HashSet<Coche> getLista() { return lista; }

    public void setLista(HashSet<Coche> lista) { no usages
        this.lista = lista;
    }
    public void anadirCoche(){ 3 usages
        System.out.println ("Introduce marca y kilométros del coche");
        Scanner scanner=new Scanner ( System.in );
        lista.add ( new Coche ( scanner.next (),scanner.nextInt () ) );
    }
    public void mostrarMarca(String marca){ 1 usage
        Iterator<Coche>miIterator=lista.iterator ();
        while(miIterator.hasNext ()){
            Coche coche=miIterator.next ();
            if(coche.getMarca ().equalsIgnoreCase ( marca )){
                System.out.println (coche);
            }


        }
    }
```

```java
public class Main {
    public static void main(String[] args) {

/*
6.- Escribir un programa Java que crea un HashSet de Objetos de tipo Coche.
El programa pide por teclado los datos de los coches y los guarda en el HashSet.
 A continuación utilizará un iterator para mostrar por pantalla lo siguiente:

•   Todos los coches introducidos.
•   Todos los coches de una marca determinada.
•   Todos los coches con menos de un número determinado de Kilómetros.
•   El coche con mayor número de Kilómetros.
•   Todos los coches ordenados por número de kilómetros de menor a mayor.

 */

        LIstaCoches milista=new LIstaCoches ();

        milista.anadirCoche ();
        milista.anadirCoche ();
        milista.anadirCoche ();
        System.out.println (milista);//mostrar coches
        milista.mostrarMarca ( "Renault" );//mostrar marca
        milista.menorkmDeterminado ( 100000 );//menor de km determinados
        milista.ordenarKm ();

        milista.ordenarKm ();
        System.out.println (milista.mayornumkm ());
```

```
C:\Users\danip\.jdks\openjdk-23\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.2.3\lib\idea_rt.jar=53109:C:\Program Files\JetBrains\
Introduce marca y kilométros del coche
renault
125000
Introduce marca y kilométros del coche
mini
40000
Introduce marca y kilométros del coche
bmw
56000
 [Coche{marca='renault', kilometros=125000}, Coche{marca='mini', kilometros=40000}, Coche{marca='bmw', kilometros=56000}]
Coche{marca='renault', kilometros=125000}
 Coche con menor kilometros que 100000 Coche{marca='mini', kilometros=40000}
 Coche con menor kilometros que 100000 Coche{marca='bmw', kilometros=56000}
Coche{marca='mini', kilometros=40000}
Coche{marca='bmw', kilometros=56000}
Coche{marca='renault', kilometros=125000}
Coche{marca='mini', kilometros=40000}
Coche{marca='bmw', kilometros=56000}
Coche{marca='renault', kilometros=125000}
Coche{marca='renault', kilometros=125000}

Process finished with exit code 0
```

Ejercicio 7

```java
public class Cadenas {
    private ArrayList<String>lista;

    public Cadenas() {
        this.lista = new ArrayList<> ();
    }

    public ArrayList<String> getLista() {
        return lista;
    }

    public void setLista(ArrayList<String> lista) {
        this.lista = lista;
    }
    public void leerArray(ArrayList<String > cadenas){
        System.out.println ("Introduce palabras en el array hasta escribir FIN para terminar");
        Scanner scan=new Scanner ( System.in );
        String entrada= scan.next ();
        while(!entrada.equalsIgnoreCase ( anotherString: "fin" )){
            cadenas.add ( entrada );
            System.out.println ("Introduce una nueva palabra");
             entrada= scan.next ();
        }
    }
    public void cadenaMasLarga(ArrayList<String>cadenas){
        int tamano=0;
        String palabra="";
        for (int i = 0; i < cadenas.size (); i++) {
            if(cadenas.get ( i ).length ()>tamano){
                tamano=cadenas.get ( i ).length ();
                palabra=cadenas.get ( i );
```

```java
public class Cadenas {
    public void leerArray(ArrayList<String > cadenas){
            System.out.println ("Introduce una nueva palabra");
             entrada= scan.next ();
        }
    }
    public void cadenaMasLarga(ArrayList<String>cadenas){
        int tamano=0;
        String palabra="";
        for (int i = 0; i < cadenas.size (); i++) {
            if(cadenas.get ( i ).length ()>tamano){
                tamano=cadenas.get ( i ).length ();
                palabra=cadenas.get ( i );
            }
        }

        System.out.println ("La cadena más larga es "+palabra);
    }

    @Override
    public String toString() {
        return "Cadenas{" +
                "lista=" + lista +
                '}';
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

/*
7.- Crea un programa en Java para calcular el String de mayor longitud de todos los contenidos en un ArrayList de String.
 El programa utilizará los siguientes métodos:
•   Método leerArray(): este método recibe como parámetro el arrayList de Strings vacio.
      El método pide por teclado cadenas de caracteres y las añade al ArrayList. La lectura de cadenas termina cuando
            se introduce la palabra "FIN".
•   Método cadenaMasLarga():este método recibe como parámetro el ArrayList de Strings con todas las cadenas leidas
     anteriormente y devuelve el String de mayor longitud.

 */
Cadenas micadena=new Cadenas ();

        ArrayList<String>mispalabras=new ArrayList<> ();

        micadena.leerArray ( mispalabras );
        micadena.cadenaMasLarga ( mispalabras );



    }
}
```

```java
public class Main {
    public static void main(String[] args) {
 */
Cadenas micadena=new Cadenas ();

        ArrayList<String>mispalabras=new ArrayList<> ();

        micadena.leerArray ( mispalabras );
```

```
C:\Users\danip\.jdks\openjdk-23\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.2.3\lib\idea_rt.jar=53201:C:\Program Files\JetBrains\I
Introduce palabras en el array hasta escribir FIN para terminar
hola
Introduce una nueva palabra
adios
Introduce una nueva palabra
antes
Introduce una nueva palabra
despues
Introduce una nueva palabra
fin
La cadena más larga es despues

Process finished with exit code 0
```